

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT : SCALA FOR DATA SCIENCE

Practical No : 3, 4, 5, 6

Aim :

3. Write a program to compute frequency distribution and cumulative frequency of a dataset.
4. Sort a dataset by a specific column and extract the top 5 rows.
5. Implement linear regression using Breeze. Fit a model to a small dataset and predict a value.
6. Perform logistic regression using Breeze. Classify a dataset with binary labels.

CODE :

```
import scala.io.Source
```

```
object omwala_M2_pract3 {
```

```
  def main(args: Array[String]): Unit = {
```

```
    println("Om Wala S119")
    println("Frequency Distribution and Cumulative Frequency")
    println("-----")
```

```
    val fileName = "/fifa.csv"
```

```
    val columnName = "ARG"
```

```
    try {
```

```
      val stream = getClass.getResourceAsStream(fileName)
```

```
      if (stream == null) {
        println(s"File '$fileName' not found in resources folder.")
        return
      }
```

```
      val source = Source.fromInputStream(stream)
```

```
      val lines = source.getLines().toList
```

```
      val header = lines.head.split(",").map(_.trim)
```

```
      val columnIndex = header.indexOf(columnName)
```

```
      if (columnIndex == -1) {
        println(s"Column '$columnName' not found in CSV file.")
      } else {
```

```
        val data = lines.tail
          .map(_.split(",").map(_.trim))
```

Name : Om Wala

Roll No : S119

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT : SCALA FOR DATA SCIENCE

```
.filter(row => row.length > columnIndex)
.flatMap(row => row(columnIndex).toDoubleOption)

val frequency = data
  .groupBy(identity)
  .view
  .mapValues(_._2.size)
  .toMap

println()
println(f"${"Value"}%-15s${"Frequency"}%-15s${"Cumulative Frequency"}")
println("-----")

var cumulativeFrequency = 0

frequency.toSeq.sortBy(_._1).foreach {
  case (value, freq) =>

    cumulativeFrequency += freq

    println(
      f"$value%-15.2f$freq%-15d$cumulativeFrequency"
    )
}

println("-----")
println(s"Total Observations: ${data.size}")
}

source.close()

} catch {
  case e: Exception =>
    println("Error: " + e.getMessage)
}
}
```

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT : SCALA FOR DATA SCIENCE

OUTPUT :

```
Run omwala_M2_pract3 x
C:\Program Files\Java\jdk-21.0.11\bin\java.exe" ...
Om Wala S119
Frequency Distribution and Cumulative Frequency
-----
Value          Frequency    Cumulative Frequency
-----
1.00           26           26
2.00           32           58
3.00           53           111
4.00           26           137
5.00           38           175
6.00           26           201
7.00           23           224
8.00           18           242
9.00           13           255
10.00          12           267
11.00           2           269
12.00           1           270
13.00           3           273
15.00           1           274
16.00           1           275
17.00           3           278
18.00           1           279
19.00           3           282
21.00           1           283
22.00           2           285
24.00           1           286
-----
Total Observations: 286
```

CODE :

```
import scala.io.Source

object omwala_M2_pract4 {

  def main(args: Array[String]): Unit = {

    println("Om Wala S119")
    println("Top 5 Rows After Sorting FIFA Dataset")
    println("-----")

    val fileName = "/fifa.csv"

    val sortColumn = "ARG"

    try {
```

Name : Om Wala
Roll No : S119

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT : SCALA FOR DATA SCIENCE

```
val stream = getClass.getResourceAsStream(fileName)

if (stream == null) {
  println("CSV file not found in resources folder.")
  return
}

val source = Source.fromInputStream(stream)

val lines = source.getLines().toList

val headers = lines.head.split(",").map(_._trim)

val columnIndex = headers.indexOf(sortColumn)

if (columnIndex == -1) {
  println(s"Column '$sortColumn' not found.")
  return
}

val data = lines.tail
  .map(_._split(",").map(_._trim))
  .filter(row => row.length > columnIndex)
  .filter(row => row(columnIndex).toDoubleOption.isDefined)

val sortedData = data.sortBy(
  row => -row(columnIndex).toDouble
)

val top5 = sortedData.take(5)

println()
println(s"Sorted by column: $sortColumn")
println("Order: Descending")
println()

println(headers.mkString(" | "))
println("-" * 80)

top5.foreach { row =>
  println(row.mkString(" | "))
}

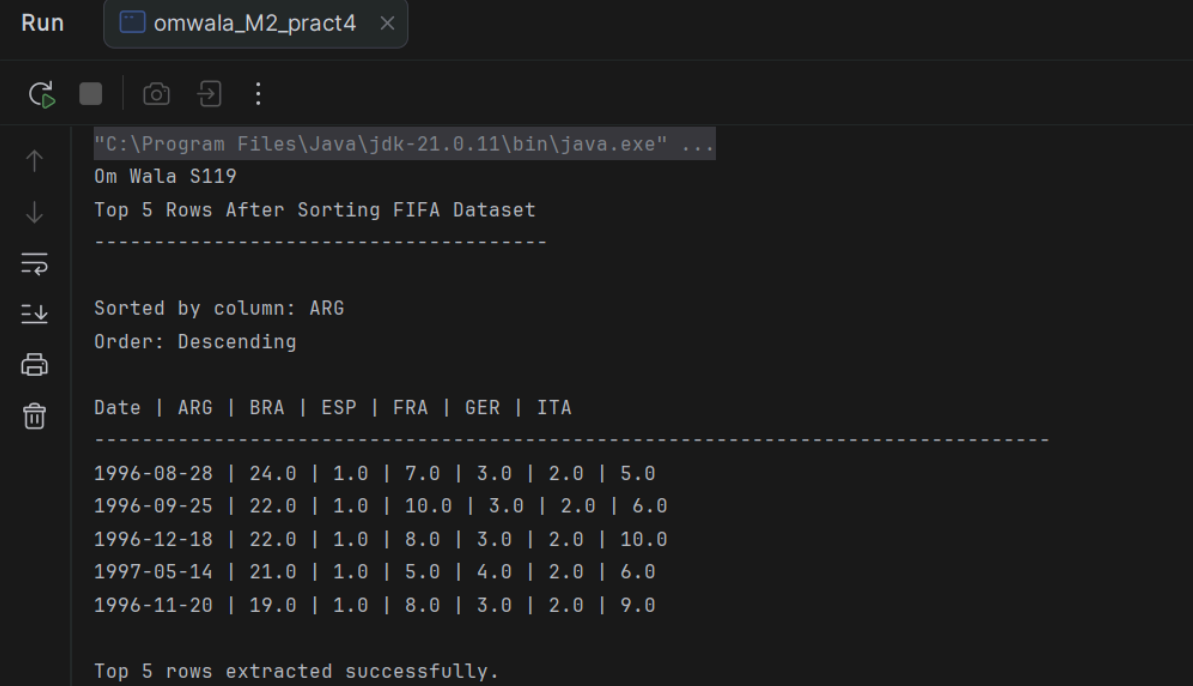
println()
println("Top 5 rows extracted successfully.")
```

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT : SCALA FOR DATA SCIENCE

```
source.close()

} catch {
  case e: Exception =>
    println("Error: " + e.getMessage)
}
}
```

OUTPUT :



```
Run omwala_M2_pract4 x
"C:\Program Files\Java\jdk-21.0.11\bin\java.exe" ...
Om Wala S119
Top 5 Rows After Sorting FIFA Dataset
-----

Sorted by column: ARG
Order: Descending

Date | ARG | BRA | ESP | FRA | GER | ITA
-----
1996-08-28 | 24.0 | 1.0 | 7.0 | 3.0 | 2.0 | 5.0
1996-09-25 | 22.0 | 1.0 | 10.0 | 3.0 | 2.0 | 6.0
1996-12-18 | 22.0 | 1.0 | 8.0 | 3.0 | 2.0 | 10.0
1997-05-14 | 21.0 | 1.0 | 5.0 | 4.0 | 2.0 | 6.0
1996-11-20 | 19.0 | 1.0 | 8.0 | 3.0 | 2.0 | 9.0

Top 5 rows extracted successfully.
```

CODE :

```
import scala.io.Source
import breeze.linalg._

object omwala_M2_pract5 {

  def main(args: Array[String]): Unit = {

    println("Om Wala S119")
    println("Linear Regression using Breeze")
    println("-----")

    val fileName = "/fifa.csv"

    val xColumn = "ARG"
    val yColumn = "ESP"

    try {
```

Name : Om Wala
Roll No : S119

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT : SCALA FOR DATA SCIENCE

```
val stream = getClass.getResourceAsStream(fileName)
```

```
if (stream == null) {  
    println("CSV file not found in resources folder.")  
    return  
}
```

```
val source = Source.fromInputStream(stream)
```

```
val lines = source.getLines().toList
```

```
val headers = lines.head.split(",").map(_.trim)
```

```
val xIndex = headers.indexOf(xColumn)  
val yIndex = headers.indexOf(yColumn)
```

```
if (xIndex == -1 || yIndex == -1) {  
    println("Required column not found.")  
    return  
}
```

```
val data = lines.tail  
    .map(_.split(",").map(_.trim))  
    .filter(row =>  
        row.length > math.max(xIndex, yIndex) &&  
        row(xIndex).toDoubleOption.isDefined &&  
        row(yIndex).toDoubleOption.isDefined  
    )  
    .map(row =>  
        (row(xIndex).toDouble, row(yIndex).toDouble)  
    )
```

```
val smallData = data.take(10)
```

```
val x = DenseVector(smallData.map(_._1).toArray)  
val y = DenseVector(smallData.map(_._2).toArray)
```

```
val xMean = sum(x) / x.length  
val yMean = sum(y) / y.length
```

```
val numerator = sum((x - xMean) * (y - yMean))  
val denominator = sum((x - xMean) * (x - xMean))
```

```
val slope = numerator / denominator
```

```
val intercept = yMean - slope * xMean
```

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT : SCALA FOR DATA SCIENCE

```
println()
println("Model Results")
println("-----")
println(f"Slope   : $slope%.4f")
println(f"Intercept : $intercept%.4f")

println()
println(s"Regression Equation:")
println(f"$yColumn = $intercept%.4f + $slope%.4f * $xColumn")

val predictionX = x(0)

val predictionY = intercept + slope * predictionX

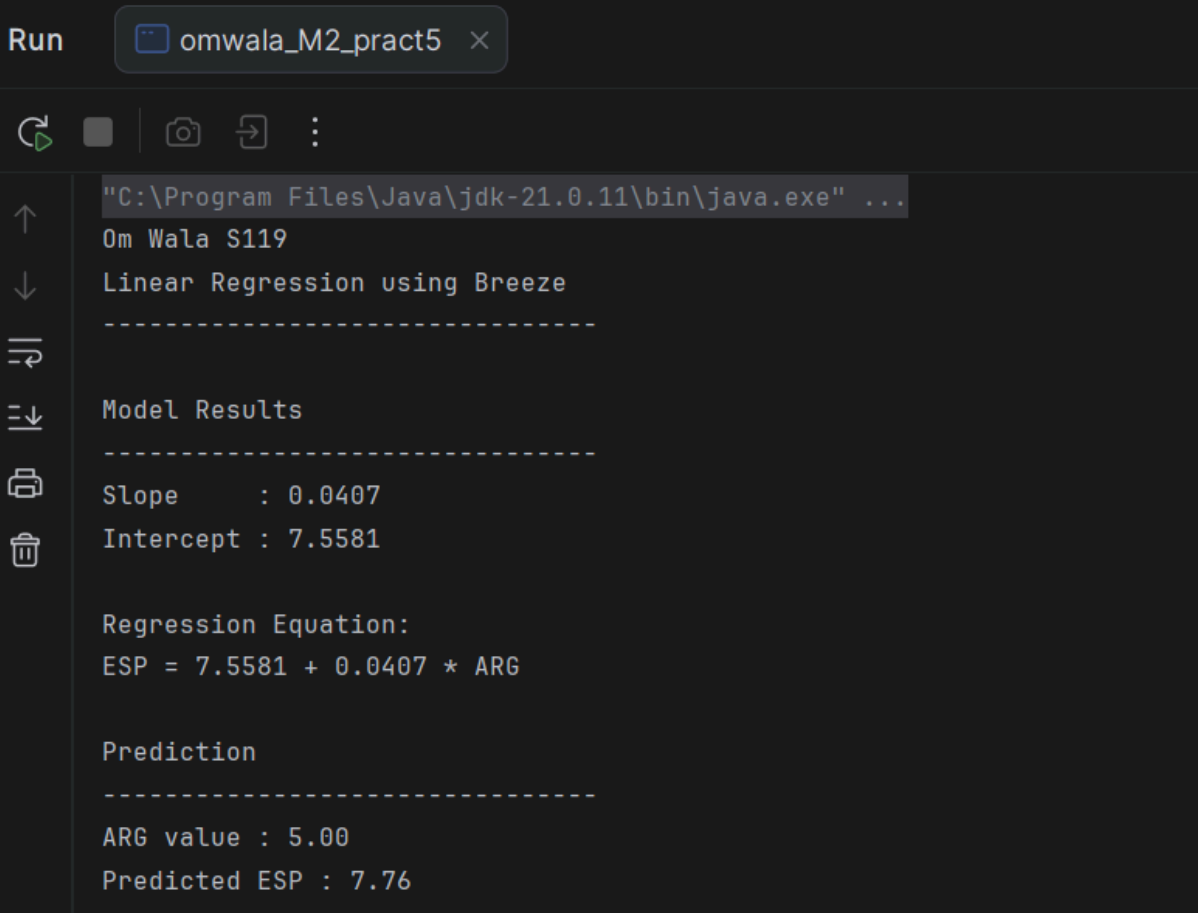
println()
println("Prediction")
println("-----")
println(f"$xColumn value : $predictionX%.2f")
println(f"Predicted $yColumn : $predictionY%.2f")

source.close()

} catch {
  case e: Exception =>
    println("Error: " + e.getMessage)
}
}
```

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT : SCALA FOR DATA SCIENCE

OUTPUT :



```
Run omwala_M2_pract5 x
"C:\Program Files\Java\jdk-21.0.11\bin\java.exe" ...
Om Wala S119
Linear Regression using Breeze
-----
Model Results
-----
Slope : 0.0407
Intercept : 7.5581

Regression Equation:
ESP = 7.5581 + 0.0407 * ARG

Prediction
-----
ARG value : 5.00
Predicted ESP : 7.76
```

CODE :

```
import scala.io.Source
import breeze.linalg._

object omwala_M2_pract6 {

  def sigmoid(z: Double): Double = {
    1.0 / (1.0 + math.exp(-z))
  }

  def main(args: Array[String]): Unit = {

    println("Om Wala S119")
    println("Logistic Regression using Breeze")
    println("-----")

    val fileName = "/fifa.csv"

    val featureColumn = "ARG"
    val targetColumn = "ESP"
```

Name : Om Wala
Roll No : S119

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT : SCALA FOR DATA SCIENCE

```
try {

    val stream = getClass.getResourceAsStream(fileName)

    if (stream == null) {
        println("CSV file not found in resources folder.")
        return
    }

    val source = Source.fromInputStream(stream)

    val lines = source.getLines().toList

    val headers = lines.head.split(",").map(_.trim)

    val featureIndex = headers.indexOf(featureColumn)
    val targetIndex = headers.indexOf(targetColumn)

    if (featureIndex == -1 || targetIndex == -1) {
        println("Required column not found.")
        return
    }

    val data = lines.tail
        .map(_.split(",").map(_.trim))
        .filter(row =>
            row.length > math.max(featureIndex, targetIndex) &&
            row(featureIndex).toDoubleOption.isDefined &&
            row(targetIndex).toDoubleOption.isDefined
        )
        .map(row =>
            (row(featureIndex).toDouble, row(targetIndex).toDouble)
        )

    val smallData = data.take(20)

    val targetValues = smallData.map(_._2).sorted
    val median = targetValues(targetValues.length / 2)

    val xData = smallData.map(_._1)
    val labels = smallData.map { case (_, y) =>
        if (y >= median) 1.0 else 0.0
    }

    val x = DenseVector(xData.toArray)
    val y = DenseVector(labels.toArray)

    var weight = 0.0
```

Name : Om Wala
Roll No : S119

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT : SCALA FOR DATA SCIENCE

```
var bias = 0.0

val learningRate = 0.01
val iterations = 1000

for (_ <- 0 until iterations) {

  var weightGradient = 0.0
  var biasGradient = 0.0

  for (i <- 0 until x.length) {

    val prediction = sigmoid(weight * x(i) + bias)

    val error = prediction - y(i)

    weightGradient += error * x(i)
    biasGradient += error
  }

  weightGradient /= x.length
  biasGradient /= x.length

  weight -= learningRate * weightGradient
  bias -= learningRate * biasGradient
}

println()
println("Model Results")
println("-----")
println(f"Weight : $weight%.4f")
println(f"Bias : $bias%.4f")
println(f"Median used for classification : $median%.2f")

println()
println("Classification Results")
println("-----")

for (i <- 0 until x.length) {

  val probability =
    sigmoid(weight * x(i) + bias)

  val predictedClass =
    if (probability >= 0.5) 1 else 0

  println(
    f"ARG = ${x(i)}%.2f " +
```

Name : Om Wala
Roll No : S119

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT : SCALA FOR DATA SCIENCE

```
f"Actual = ${y(i).toInt} " +  
f"Probability = $probability%.4f " +  
s"Predicted = $predictedClass"  
)  
}  
  
val newValue = x(0)  
  
val probability =  
  sigmoid(weight * newValue + bias)  
  
val prediction =  
  if (probability >= 0.5) 1 else 0  
  
println()  
println("New Prediction")  
println("-----")  
println(f"ARG value      : $newValue%.2f")  
println(f"Probability    : $probability%.4f")  
println(s"Predicted class : $prediction")  
  
source.close()  
  
} catch {  
  case e: Exception =>  
    println("Error: " + e.getMessage)  
}  
}  
}
```

SHETH L.U.J AND SIR M.V COLLEGE
SUBJECT : SCALA FOR DATA SCIENCE

OUTPUT :

```
Run omwala_M2_pract6 x
[C:\Program Files\Java\jdk-21.0.11\bin\java.exe" ...
Om Wala S119
Logistic Regression using Breeze
-----
Model Results
-----
Weight : 0.0439
Bias   : -0.1200
Median used for classification : 6.00

Classification Results
-----
ARG = 5.00  Actual = 1  Probability = 0.5248  Predicted = 1
ARG = 12.00 Actual = 1  Probability = 0.6003  Predicted = 1
ARG = 9.00  Actual = 1  Probability = 0.5683  Predicted = 1
ARG = 9.00  Actual = 1  Probability = 0.5683  Predicted = 1
ARG = 8.00  Actual = 0  Probability = 0.5575  Predicted = 1
ARG = 9.00  Actual = 1  Probability = 0.5683  Predicted = 1
ARG = 8.00  Actual = 1  Probability = 0.5575  Predicted = 1
ARG = 10.00 Actual = 1  Probability = 0.5791  Predicted = 1
ARG = 6.00  Actual = 1  Probability = 0.5358  Predicted = 1
ARG = 8.00  Actual = 0  Probability = 0.5575  Predicted = 1
ARG = 9.00  Actual = 1  Probability = 0.5683  Predicted = 1
ARG = 9.00  Actual = 1  Probability = 0.5683  Predicted = 1
ARG = 9.00  Actual = 1  Probability = 0.5683  Predicted = 1
ARG = 9.00  Actual = 0  Probability = 0.5683  Predicted = 1
ARG = 10.00 Actual = 0  Probability = 0.5791  Predicted = 1
ARG = 7.00  Actual = 0  Probability = 0.5467  Predicted = 1
ARG = 8.00  Actual = 0  Probability = 0.5575  Predicted = 1
ARG = 8.00  Actual = 0  Probability = 0.5575  Predicted = 1

ARG = 10.00 Actual = 0  Probability = 0.5791  Predicted = 1
ARG = 7.00  Actual = 0  Probability = 0.5467  Predicted = 1
ARG = 8.00  Actual = 0  Probability = 0.5575  Predicted = 1
ARG = 8.00  Actual = 0  Probability = 0.5575  Predicted = 1
ARG = 10.00 Actual = 0  Probability = 0.5791  Predicted = 1
ARG = 5.00  Actual = 0  Probability = 0.5248  Predicted = 1

New Prediction
-----
ARG value      : 5.00
Probability    : 0.5248
Predicted class : 1

Process finished with exit code 0
```

Name : Om Wala
Roll No : S119